

COMP0078: Supervised Learning - Lecture Notes

Comprehensive Course Notes

Contents

1	Linear Regression Fundamentals	6
1.1	Mean Squared Error Objective	6
1.2	Excess Risk and Risk Decomposition	6
1.3	Bayes Risk and Optimal Estimation	6
1.4	Squared Loss and Conditional Mean	6
1.5	Bias-Variance Decomposition	7
1.6	No Free Lunch Theorem	7
1.7	Linear Regression on Feature-Mapped Variables	8
1.8	Pseudoinverse	8
1.9	Kernel Functions and Positive Semi-Definite (PSD) Property	8
1.10	Instability with Small Singular Values	8
2	Regularization and Projections	9
2.1	Ridge Regression Solution	9
2.2	Eigenvalue Perturbation	9
2.3	Orthogonal Projectors	9
2.4	Projection onto Column Space	9
2.5	Projection onto Hyperplanes	10
2.6	Support Vector Machines: Optimal Separating Hyperplane	10
3	Kernel Methods and Advanced SVM	12
3.1	Dual Problem Formulation	12
3.2	Hyperparameter Selection	12
3.3	SVM with Kernel	12
3.4	SVM Regression	12
3.5	Scaling and Margin in SVM	13
3.6	Large-Scale SVM: Primal vs Dual	13
4	Logistic Regression and Classification	13
4.1	Logistic Regression for Binary Classification	13
4.2	Binary Classification Loss	13
4.3	Logistic Regression Formulation	13
4.4	Quadratic Upper Bound	13
4.5	Gradient Descent	13
4.6	M-Smoothness and Gradient Lipschitz Continuity	14

5	Convexity, Smoothness, and Optimization Theory	16
5.1	Differentiability and Convexity	16
5.2	Convexity and Smoothness Bounds	16
5.3	Gradient Descent Convergence	16
5.4	Square Loss Lipschitz Property	16
5.5	Regularized Regression	16
5.6	Subgradients and Non-Smooth Functions	16
5.7	Subgradient Descent Method	17
6	Tree-Based Methods	17
6.1	Tree Partition Functions	17
6.2	Regression Trees	17
6.3	Cost-Complexity Pruning	17
6.4	Classification Trees	18
7	Ensemble Methods and Boosting	19
7.1	Classification Tree Impurity Measures	19
7.2	Chernoff Bound	19
7.3	Bagging Algorithm	19
7.4	Random Forests	20
7.5	Boosting Algorithm	20
7.6	Adaboost	20
7.7	Adaboost Training Error Bound	21
7.8	Training Error Analysis	21
7.9	Boosting as Forward Stagewise Additive Model	21
7.10	Typical Loss Functions	22
7.11	Bagging vs Boosting	22
8	Theoretical Foundations: Empirical Risk and Generalization	23
8.1	Empirical Risk and Its Properties	23
8.2	Excess Risk Decomposition	23
8.3	Empirical Risk Minimization on Finite Hypothesis Spaces	24
8.4	Hoeffding's Inequality and High-Probability Bounds	24
8.5	Union Bound and Sample Complexity	24
8.6	Gap When True Function Outside Hypothesis Class	25
9	Rademacher Complexity and Advanced Generalization Bounds	26
9.1	Regularization and Excess Risk Decomposition	26
9.2	Rademacher Complexity	26
9.3	Generalization Bounds via Rademacher Complexity	27
9.4	Lipschitz Contraction and Bounded Losses	27
10	McDiarmid Inequality, RKHS, and Optimization Methods	28
10.1	McDiarmid's Inequality and High-Probability Bounds	28
10.2	Gradient Descent Convergence	28
10.3	Reproducing Kernel Hilbert Space (RKHS) and Bounded Kernels	29
10.4	Ivanov Regularization	29
10.5	Projected Gradient Descent (PGD)	29
10.6	Introduction to Online Learning	29

11 Online Learning with Experts	30
11.1 Online Learning with Experts Framework	30
11.2 Weighted Majority Algorithm	30
11.3 Loss Functions for Online Learning	31
11.4 Weighted Average Algorithm	31
12 Advanced Online Learning Algorithms	33
12.1 HEDGE Algorithms	33
12.2 Perceptron Algorithm	33
12.3 Online Gradient Descent with Hinge Loss	34
12.4 Surrogate Loss Approaches	34
13 Multi-Class Classification and Surrogates	36
13.1 Multi-Class Classification	36
13.2 Finite Dimensional Loss as Matrix	36
13.3 Implicit Embeddings	36
13.4 Using Implicit Embeddings	37
13.5 General Surrogate Approach	37
13.6 Kernel Feature Approximation	37
14 Convergence Analysis and Consistency	38
14.1 Decoding in Multi-Class	38
14.2 Implicit Embeddings Condition	38
14.3 Universal Consistency	38
14.4 Learning Rates	38

Introduction

These lecture notes cover the theory and practice of supervised learning, where the goal is to learn a function that maps input features to output targets based on a labeled dataset.

Course Structure:

1. **Foundations (Sections 1-3):** Core supervised learning problems: regression, binary classification, and multi-class classification
2. **Optimization & Regularization (Sections 4-6):** Optimization theory, regularization techniques, and convergence analysis
3. **Advanced Methods (Sections 7-9):** Kernel methods, tree-based models, ensemble learning
4. **Statistical Learning Theory (Sections 10-13):** Generalization bounds, complexity measures, consistency results
5. **Online Learning (Sections 14-15):** Sequential learning in adversarial settings with experts and online algorithms

The course develops both the practical algorithms used in machine learning and the theoretical foundations that explain when and why they work.

Part I: Foundations — Learning Paradigms

We begin with the three core supervised learning problems: regression (continuous outputs), binary classification, and multi-class classification. These form the foundation for all subsequent methods.

1 Linear Regression Fundamentals

1.1 Mean Squared Error Objective

The most fundamental supervised learning problem is linear regression, which seeks a linear function to predict continuous outputs. The squared loss between predictions and targets is:

$$L(w) = \|Xw - y\|^2 = 2X^T Xw - 2X^T y$$

where:

- $X \in \mathbb{R}^{n \times d}$ is the design matrix (data)
- $w \in \mathbb{R}^d$ is the weight vector
- $y \in \mathbb{R}^n$ is the target vector

1.2 Excess Risk and Risk Decomposition

Definition 1.1 (Excess Risk). The excess risk is the difference between the expected loss of a learned function and the Bayes-optimal risk (the best achievable loss):

$$\mathcal{E}(f_S) - \inf \mathcal{E}(f) = \mathbb{E}_{(x,y) \sim D}[\ell(f_S(x), y)] - \mathbb{E}_{(x,y) \sim D}[\ell(f^*(x), y)]$$

Our goal in supervised learning is to minimize excess risk, which goes to zero as $n \rightarrow \infty$ (with a sufficiently large sample dataset).

1.3 Bayes Risk and Optimal Estimation

Definition 1.2 (Bayes Estimator). The Bayes estimator minimizes the posterior expected loss (pointwise maximizer):

$$f^*(x) = \arg \min_y \mathbb{E}_{y \sim p(y|x)}[\ell(\hat{y}, y)]$$

For squared loss $\ell(y, \hat{y}) = (y - \hat{y})^2$, this gives the conditional mean:

$$f^*(x) = \mathbb{E}[Y|X = x] = \text{conditional expectation}$$

and with 0-1 loss, the Bayes classifier predicts the mode:

$$f^*(x) = \arg \max_y P(Y = y|X = x)$$

1.4 Squared Loss and Conditional Mean

Definition 1.3 (Squared Loss). The squared loss for regression is:

$$\ell(x) = \int y dP(y|x) = \mathbb{E}[Y|X]$$

The 0-1 loss is the mode of $P(y|x)$, giving the most likely class.

1.5 Bias-Variance Decomposition

A fundamental insight in supervised learning is that the excess risk can be decomposed into bias and variance terms. Let $\bar{f}_S$ denote the averaged model trained on n points.

Proposition 1.1 (Bias-Variance Decomposition). The excess risk decomposes as:

$$\mathbb{E}_S[(\bar{f}_S(x) - f^*(x))^2] = (f_n(x) - f^*(x))^2 + \mathbb{E}_S[(\bar{f}_S(x) - f_n(x))^2]$$

where:

- **Bias:** $\mathbb{E}_S[\ell(f_S(x)) - \ell(f^*(x))] = \|f_n - f^*\|_{L^2(\rho_X)}$ — expected distance from the true function (measures how well the function class fits the underlying truth)
- **Variance:** $\mathbb{E}_S[(\bar{f}_S(x) - f_n(x))^2]$ — variance of model outputs when trained on different sampled datasets (measures sensitivity to the specific training data)

The decomposition shows that when $\bar{f}_n(x) = \mathbb{E}_S[f_S(x)]$ is the averaged model trained on n points, then:

$$\mathbb{E}_S[(\bar{f}_S(x) - f^*(x))^2] = (\bar{f}_n(x) - f^*(x))^2 + \mathbb{E}_S[(\bar{f}_S(x) - \bar{f}_n(x))^2]$$

Thus, the total error decomposes as bias plus variance, where:

$$\text{Bias} = \mathbb{E}_S[\ell(f_S(x)) - \ell(f^*(x))] = \|f_n - f^*\|_{L^2(\rho_X)}$$

$$\text{Variance} = \mathbb{E}_S[(\bar{f}_S(x) - \bar{f}_n(x))^2]$$

where the second term measures the variance of the output when trained on different randomized data.

1.6 No Free Lunch Theorem

Theorem 1.1 (No Free Lunch Theorem for Binary Classification). Let algorithm $\mathcal{A}$ be a learning algorithm for binary classification. If $n < |X|/2$, then there exists a distribution ρ over $X \times Y$ such that:

1. There exists a Bayes estimator $f_n^* : X \rightarrow Y$ (with $\mathbb{E}[\ell(f_n^*(x), y)] = 0$)
2. For any model $A(S) = f_S$ trained on a dataset S of n independent samples:

$$\mathbb{P}_S(P_{\rho, y}[\mathcal{A}(x), f_S(x) \neq y] > \frac{1}{2}) > \frac{1}{4}$$

On at least 1 in 7 random training sets, the algorithm's test error is $> 12.5\%$ regardless of n .

If one doesn't make assumptions on labels of points, the learner has no better than random guessing.

1.7 Linear Regression on Feature-Mapped Variables

Definition 1.4 (Feature-Mapped Linear Regression). We can extend linear regression by mapping features to a higher-dimensional space:

$$\begin{aligned}\text{Primal solution: } \hat{w} &= (\Phi^T \Phi)^{-1} \Phi^T y \quad \text{--- cost } O(D^3 + nD^2) \\ \text{Dual solution: } \hat{w} &= \Phi^T (\Phi \Phi^T)^{-1} y \quad \text{--- cost } O(n^3 + Dn^2)\end{aligned}$$

where $\Phi \in \mathbb{R}^{n \times D}$ is the feature-mapped data matrix with feature dimension D .

1.8 Pseudoinverse

Definition 1.5 (Pseudoinverse). The pseudoinverse (Moore-Penrose inverse) of a matrix $\Phi \in \mathbb{R}^{n \times D}$ is denoted $\Phi^\dagger$ and defined as:

$$\Phi \Phi^\dagger \Phi = \Phi \quad \text{and} \quad \Phi^\dagger \Phi \Phi^\dagger = \Phi^\dagger$$

For full rank Φ , $\Phi^\dagger = (\Phi^T \Phi)^{-1} \Phi^T$ (left pseudoinverse when $n > D$).

1.9 Kernel Functions and Positive Semi-Definite (PSD) Property

Definition 1.6 (Kernel Function). A kernel function k is an inner product $\Rightarrow k$ is PSD (positive semi-definite with $k \succeq 0$).

However, the converse $k \succeq 0 \Rightarrow k$ is a kernel (inner product) is not generally true. We need to have $k \succeq 0$ for any finite set on X (Moore-Aronszajn theorem).

1.10 Instability with Small Singular Values

Remark 1.1 (Instability of Pseudoinverse). The pseudoinverse $\Phi^\dagger$ can become numerically unstable when Φ has small singular values. If $\Phi = U \Sigma V^T$ with smallest nonzero singular value σ_r very small, then:

$$\Phi^\dagger = V \Sigma^{-1} U^T$$

The inverse of small singular values becomes very large, amplifying noise in the prediction. This motivates the use of regularization (ridge regression) to stabilize the solution.

2 Regularization and Projections

2.1 Ridge Regression Solution

Definition 2.1 (Ridge Regression with ℓ_2 Regularization). Ridge regression adds ℓ_2 regularization to prevent overfitting from small singular values:

$$\hat{w}_\lambda = \arg \min_w \left[\frac{1}{n} \sum_{i=1}^n (\Phi w - y)_i^2 + \lambda \|w\|^2 \right]$$

The closed-form solution is:

$$\hat{w}_\lambda = (\Phi^T \Phi + \lambda I)^{-1} \Phi^T y$$

This always exists since $\Phi^T \Phi + \lambda I$ is positive definite for $\lambda > 0$.

2.2 Eigenvalue Perturbation

Definition 2.2 (Eigenvalue Perturbation). If $A + \lambda I$ has eigenvalues of $\mu_i + \lambda$, where μ_i is the eigenvalue of A .

2.3 Orthogonal Projectors

Definition 2.3 (Orthogonal Projector). A matrix $P \in \mathbb{R}^{n \times n}$ is an orthogonal projector if:

$$P^2 = P \quad \text{and} \quad P = P^T$$

Definition 2.4 (Complementary Projector). Given an orthogonal projector $P \in \mathbb{R}^{m \times n}$, the associated complementary projector is $I - P$.

Remark 2.1 (Orthogonality Condition). For complementary projectors:

$$(I - P)^T \ell(Pw) = 0 \quad \forall w \in \mathbb{R}^d$$

Definition 2.5 (Decomposition via Projectors). Any vector can be decomposed as:

$$x = Px + (I - P)x$$

where:

- $Px \in V$ — the component in the subspace
- $(I - P)x \in V^\perp$ — the component in the orthogonal complement

2.4 Projection onto Column Space

Definition 2.6 (Orthogonal Projection onto Column Space). Given a matrix $V \in \mathbb{R}^{n \times k}$ with orthonormal columns:

$$P = VV^T \quad \text{is the orthogonal projector onto its column space}$$

2.5 Projection onto Hyperplanes

Definition 2.7 (Projection onto Hyperplane). The projection matrix onto the hyperplane $w^T x + b = 0$ is:

$$P = I - \frac{ww^T}{\|w\|^2}$$

To find the closest point x on the hyperplane w, b : if x^* , $w^T x^* + b = 0$ is a given point z :

$$P_z(w, b) = \min_{x \in \mathbb{R}^d} \|z - x\|^2 \quad \text{subject to } wx + b = 0$$

With $w^T x + b = 0 \Rightarrow w^T(x + \frac{w0}{\|w\|^2}) = 0$, so we can express:

$$x^* = \frac{w0}{\|w\|^2} = Pv = (I - \frac{ww^T}{\|w\|^2})v$$

So:

$$\min \|z + \frac{wb}{\|w\|^2} - Pv\|^2$$

The solution is the projection of $(z + \frac{wb}{\|w\|^2})$ onto the subspace $\{Pv : v \in \mathbb{R}^d\}$.

Therefore:

$$P_z(w, b) = \frac{|w^T z + b|}{\|w\|}$$

2.6 Support Vector Machines: Optimal Separating Hyperplane

Definition 2.8 (Margin). Define the margin on training point i as:

$$\rho_S(w, b) := \min_{i \in [m]} P_S(w, b), \quad \rho_S(w, b) = \frac{|\not\llcorner y_i(w^T x_i + b)|}{\|w\|}$$

The canonical hyperplane: we can choose $\|w\|$ such that $\beta(w, b) = \frac{1}{\|w\|}$, i.e., $\min_i y_i(w^T x_i + b) = 1$.

Optimal separating hyperplane: Maximize the margin:

$$\min_{w, b} \frac{1}{2} \|w\|^2 \quad (\text{or } w \in \mathbb{R}^d)$$

subject to $y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, m$.

Margin of OSH:

$$L(w, b) = \frac{1}{2} w^T w - \sum_{i=1}^m \alpha_i [y_i(w^T x_i + b) - 1]$$

where the $1/\|w\|$ is the margin of OSH.

Optimality conditions:

$$\frac{\partial L}{\partial b} = - \sum_{i=1}^m y_i \alpha_i = 0, \quad \frac{\partial L}{\partial w} = w - \sum_{i=1}^m \alpha_i y_i x_i = 0 \Rightarrow w = \sum_{i=1}^m \alpha_i y_i x_i$$

Dual problem:

$$\max_{\alpha} Q(\alpha) = -\frac{1}{2} \alpha^T A \alpha + \sum_{i \in [m]} \alpha_i$$

where $A := (y_i y_j x_i^T x_j)_{ij}$, and $(\alpha_i x_j y_j) = (\text{osH } Ax = \omega^T \omega)$.

Part III: Advanced Supervised Learning Methods

Beyond linear models and simple decision trees lie powerful techniques for handling complex, nonlinear relationships. We explore kernel methods for implicit feature spaces, tree-based methods, and ensemble approaches that combine multiple weak learners.

3 Kernel Methods and Advanced SVM

3.1 Dual Problem Formulation

Remark 3.1 (New Dual Problem). The dual optimization problem is:

$$Q(\alpha) = -\frac{1}{2}\alpha^T A \alpha + \sum_{i=1}^m \alpha_i$$

subject to $\sum_i y_i \alpha_i = 0$ and $0 \leq \alpha_i \leq C$, for $i = 1, \dots, m$.

3.2 Hyperparameter Selection

Remark 3.2 (Selection of C via Cross-Validation). The parameter $\bar{\alpha}_i$ is a piecewise quadratic function of C . The value of C is often selected by minimizing Leave-One-Out (LOO) cross-validation error.

For each C , train full data set to identify SVs. For each support vector x_i , retrain on $m - 1$ points and test on x_i . For non-SV, prediction unchanged. By KKT conditions:

Only SV can be misclassified. If one SV is unclassified (from KKT: $c_i \in \text{obj} \Rightarrow y_i(w^T x_i + b) \geq 1, \xi_i = 0$) then $y_i(w^T x_i + b) \geq 1, \xi_i = 0$ so:

$$\text{LOO error} = \frac{\# \text{ misclassified SVs}}{m} \leq \frac{\# \text{SVs}}{m}$$

3.3 SVM with Kernel

Definition 3.1 (Kernel SVM). SVM with kernel can be formulated as:

$$f(x) = \sum_i y_i \alpha_i k(x_i, x) + b, \quad x \in \mathcal{X} \quad \text{and} \quad A = (y_i y_j k(x_i, x_j))_{ij}$$

A new point $x \in \mathcal{X}$ is classified as $\text{sgn}(f(x))$.

Definition 3.2 (SVM with Kernel Loss). The empirical risk with kernel is:

$$E_\lambda(w, b) = \sum_{i=1}^m \max(1 - y_i(\langle w, \phi(x_i) \rangle + b), 0) + \lambda \|w\|^2 \quad \text{with} \quad \lambda = \frac{1}{2C}$$

SVM is $\text{CE}_{\frac{1}{2C}}(w, b) := \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \text{hinge}(y_i, \langle w, \phi(x_i) \rangle + b)$.

3.4 SVM Regression

Definition 3.3 (SVM Regression). We can choose loss $|y - f(x)|_\varepsilon = \max(|y - f(x)| - \varepsilon, 0)$ to handle regression:

$$\min_{w, b} \frac{1}{2} w^T w + C \sum_{i=1}^m (\xi_i^+ + \xi_i^-)$$

subject to:

$$\begin{aligned} w^T x_i + b - y_i &\leq \varepsilon + \xi_i^+ \\ y_i - w^T x_i - b &\leq \varepsilon + \xi_i^- \\ \xi_i^+, \xi_i^- &\geq 0, \quad i = 1, \dots, m \end{aligned}$$

3.5 Scaling and Margin in SVM

Remark 3.3 (SVM Loss Scale Sensitivity). SVM loss is scale sensitive (hinge loss $\max(1 - y_i f(x_i), 0)$). If scaling $f(x)$, SV will change. The boundary changes. The penalty C must compensate for any scaling. Recall we normalized the hyperplane such that $\min_i y_i(w^T x_i + b) = 1$. We need to adjust $\|w\|$ based on global data. Here we use C to adjust.

3.6 Large-Scale SVM: Primal vs Dual

Remark 3.4 (Size Considerations). If data size $m \gg N$ (feature size) and we cannot use the kernel trick, we should solve the primal problem directly:

$$\min_{w,b} \frac{1}{2} w^T w + C \sum_{i=1}^m \max(1 - y_i(w^T x_i + b), 0) \quad (\text{Single convex objective})$$

4 Logistic Regression and Classification

4.1 Logistic Regression for Binary Classification

Definition 4.1 (Logistic Regression). Logistic regression models the probability of the positive class:

$$\ell(x) = \log(1 + e^{-f(x)y}) \quad \text{for binary classification}$$

4.2 Binary Classification Loss

Definition 4.2 (Binary Classification Losses). A binary classification loss measures the relationship between $f(x)$ and sign of y as:

- **0-1 loss:** $\ell_0(x) = \mathbb{I}[f(x) \neq y]$ — square loss $\ell(x) = (f(x) - y)^2$
- **Hinge loss:** $\ell(x) = \max(1 - yf(x), 0)$
- **Logistic loss:** $\ell(x) = \log(1 + \exp(-yf(x)))$

4.3 Logistic Regression Formulation

Definition 4.3 (Logistic Regression Model). The logistic regression model:

$$P_\theta(x) = \frac{1}{1 + e^{-w^T f(x)}}$$

4.4 Quadratic Upper Bound

Lemma 4.1 (Quadratic Upper Bound). Let F be M -smooth, with $M > 0$, then for any $z \in \mathbb{R}^d$:

$$F(z) \leq F(w) + \nabla F(w)^T(z - w) + \frac{M}{2} \|z - w\|^2$$

4.5 Gradient Descent

Definition 4.4 (Gradient Descent). When $\eta = (\mu_k - \eta) \nabla F(\mu_k)$, with $\eta > 0$.

4.6 M-Smoothness and Gradient Lipschitz Continuity

Definition 4.5 (M-Smoothness Condition). A differentiable function $F : \mathbb{R}^d \rightarrow \mathbb{R}$ is M -smooth if its gradient is Lipschitz continuous with constant M :

$$\|\nabla F(w) - \nabla F(z)\| \leq M\|w - z\| \quad \forall w, z \in \mathbb{R}^d$$

Equivalently, the Hessian is bounded: $\|\nabla^2 F(w)\| \leq M$ everywhere.

Part II: Optimization & Regularization

When learning from finite data, we must balance fitting the training data with avoiding overfitting. This section develops optimization theory (convexity, smoothness) and regularization techniques to achieve this balance.

5 Convexity, Smoothness, and Optimization Theory

5.1 Differentiability and Convexity

Definition 5.1 (Differentiable Convex Function). Let $F : \mathbb{R}^d \rightarrow \mathbb{R}$ be differentiable. Then F is convex $\Leftrightarrow$ for all $w, z \in \mathbb{R}^d$:

$$F(z) \geq F(w) + \nabla F(w)^T(z - w)$$

5.2 Convexity and Smoothness Bounds

Definition 5.2 (Convexity-Smoothness Relationship). We have:

$$F(w) + \nabla F(w)^T(z - w) \leq F(z) \leq F(w) + \nabla F(w)^T(z - w) + \frac{M}{2}\|z - w\|^2$$

where the left inequality is **convexity** and the right is **smoothness**.

5.3 Gradient Descent Convergence

Theorem 5.1 (Gradient Descent Convergence). Let F be convex and M -smooth. Assume $\mathbb{R}^d$ and F admits a minimum in $w_* \in \mathbb{R}^d$. Let $(w_k)_{k=1}^\infty$ be a sequence produced by GD with $\eta = \frac{1}{M}$. Then:

$$F(w_k) - F(w_*) \leq \frac{M}{2\xi}\|w_0 - w_*\|^2 \quad \text{and} \quad F(w_{k+1}) \leq F(w_k) - \frac{1}{2M}\|\nabla F(w_k)\|^2$$

5.4 Square Loss Lipschitz Property

Definition 5.3 (Square Loss). The square loss $\ell(w) = (w^T x - y)^2$ has gradient:

$$\nabla \ell(w) = 2(w^T x - y)x, \quad \nabla \ell(w) - \nabla \ell(z) = 2x(x^T)^*(w - z)$$

So $\|\nabla \ell(w) - \nabla \ell(z)\|^* \leq 2\|x\|^2\|w - z\|$, which means square loss is $2\|x\|^2$ Lipschitz.

5.5 Regularized Regression

Definition 5.4 (Regularized Loss). The regularized regression loss is:

$$E_S(w) = \frac{1}{m} \sum_i (w^T x_i - y_i)^2 + \lambda \|w\|^2$$

Then $\nabla E_S(w) = 2(C_S + \lambda I)w - \frac{m}{n} \sum_i y_i x_i$, where $C_S = \frac{1}{m} \sum_{i=1}^m x_i x_i^T$.

Hence $\|\nabla E_S(w) - \nabla E_S(z)\| \leq 2\|(C_S + \lambda I)\|\|w - z\|$, where $\|(C_S + \lambda I)\|$ is the operator norm and takes the largest singular value (PCA).

5.6 Subgradients and Non-Smooth Functions

Definition 5.5 (Subgradient). For a convex function $F : \mathbb{R}^d \rightarrow \mathbb{R}$, a vector $g \in \mathbb{R}^d$ is a subgradient of F at w if for all $z \in \mathbb{R}^d$:

$$F(z) \geq F(w) + g^T(z - w)$$

The set of all subgradients at w is the subdifferential $\partial F(w)$. For differentiable functions, $\partial F(w) = \{\nabla F(w)\}$.

5.7 Subgradient Descent Method

Definition 5.6 (Subgradient Descent). For a convex (but possibly non-smooth) function F with L -Lipschitz subgradients, the subgradient descent method iterates:

$$w_{t+1} = w_t - \eta_t g_t$$

where $g_t \in \partial F(w_t)$ is any subgradient at w_t , and $\eta_t > 0$ is the step size.

With step size $\eta_t = \frac{1}{L\sqrt{t}}$, the method achieves $O(1/\sqrt{T})$ convergence rate:

$$\mathbb{E}[F(\bar{w}_T) - F(w^*)] \leq \frac{L\|w_0 - w^*\|}{\sqrt{T}}$$

where $\bar{w}_T = \frac{1}{T} \sum_{t=1}^T w_t$.

6 Tree-Based Methods

6.1 Tree Partition Functions

Definition 6.1 (Tree Partition Function). The output of a tree classification is:

$$f(x) = \sum_p c_p \mathbb{I}[x \in \mathcal{R}_p] \quad \text{for each } x, \text{ only 1 } \mathcal{R}_p \text{ is active; all others zero}$$

$$\text{Usually } c_p = \text{avg}(y_i : x_i \in \mathcal{R}_p) = \frac{\sum_{x_i \in \mathcal{R}_p} y_i}{\sum_i \mathbb{I}[x_i \in \mathcal{R}_p]}$$

6.2 Regression Trees

Definition 6.2 (Regression Trees). Define the pair of axis-parallel half-planes:

$$R_1(j, S) = \{x : x_j \leq S\}, \quad R_2(j, S) = \{x : |x_j > S\}$$

Solve the problem:

$$\min_{j, S} \left[\min_{c_1} \sum_{x_i \in R_1(j, S)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j, S)} (y_i - c_2)^2 \right], \quad j \text{ is feature, } S \text{ is half-plane}$$

Inner minimization is solved by using average in each half-plane: $\bar{c}_1 = \text{ave}(y_i : x_i \in R_1(j, S))$, $\bar{c}_2 = \text{ave}(y_i : x_i \in R_2(j, S))$.

Outer minimization can be done by searching with computation $O(nm)$ given a features and m coding points in plane.

6.3 Cost-Complexity Pruning

Remark 6.1 (Cost-Complexity Pruning). Grow a large tree $\mathcal{T}$. Look at for a subtree $T_A \subseteq \mathcal{T}$ which minimizes:

$$\text{Ca}(T) = \sum_{p \in T} m_p \Phi(T) + \lambda |T|, \quad \mathcal{R}_p(T) = \frac{1}{m_p} \sum_{x_i \in \mathcal{R}_p} (y_i - c_p)^2 \quad (\text{tree error})$$

First term increases when pruning, while second term decreases. Every time a pruned tree is chosen, c_p will change!

6.4 Classification Trees

Definition 6.3 (Classification Tree). In each region, the empirical class probabilities are $\hat{p}_{pk} = \frac{1}{m_p} \sum_{x_i \in \mathcal{R}_p} \mathbb{I}[y_i = k]$.

We classify an input in region p into the class with maximum probability:

$$f(x) = \arg \max_k \frac{\sum_{x_i: y_i=k} I[x \in \mathcal{R}_p]}{\sum_{x_i} I[x \in \mathcal{R}_p]} \quad (\text{go to region } \mathcal{R}_p \text{ first, then assign it to } \arg \max)$$

7 Ensemble Methods and Boosting

7.1 Classification Tree Impurity Measures

Definition 7.1 (Node Impurity Measures). In classification trees, we can choose different impurity measures for deciding splits:

1. **Misclassification error:** Fraction of misclassified samples in the node. Less sensitive to changes in probabilities.
2. **Gini index:** $G = \sum_{k=1}^K p_k(1 - p_k)$ where p_k is the proportion of class k in the node. Ranges from 0 (pure node) to $1 - 1/K$ (maximally impure).
3. **Cross entropy (Information gain):** $H = -\sum_{k=1}^K p_k \log p_k$. Sensitive to changes in probabilities, especially smaller frequencies.

Practical use: Gini index and cross entropy are more sensitive and preferred for tree growth. Misclassification error is often used for pruning to match the 0-1 loss on the test set.

7.2 Chernoff Bound

Definition 7.2 (Chernoff Bound). Let $X_1, X_2, \dots, X_n$ be independent random variables. Assume $\delta = \mathbb{E}[X_i]$.

Let $X := \sum X_i$ and $\mu := \mathbb{E}[X] := \sum \mathbb{E}[X_i]$. Then for any $b \in \mathbb{E}_{x,y}$:

$$P(X = k_b) \leq \exp\left(-\frac{(\mu - \delta)^2}{2\mu}\right)$$

7.3 Bagging Algorithm

Definition 7.3 (Bagging). Bagging reduces the variance of a classifier by training many variants of the classifier and then voting:

- **Training data:** $S = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\} \subset \mathbb{R}^d \times \{-1, 1\}$
- **Ensemble size** T (number of classifiers)
- **Resample size** M (usually $M = m$)
- **Classifier function** $h_S(x)$

For $t = 1$ to T do:

- $S(t) := M$ samples with replacement from S
- Train $h_{S(t)}(x)$
- $H(x) = \text{sign}\left(\sum_{t=1}^T h_{S(t)}(x)\right)$ for regression, we do average

The probability that a particular example does not appear in bag $S(t)$ is $(1 - \frac{1}{m})^m$, which is roughly 0.368. So 63% distinct examples in each $S(t)$.

7.4 Random Forests

Definition 7.4 (Random Forests). Build a tree for each split only check a subset of size k features.

k usually $\sqrt{n}$ or $\log n$. Feature means x_j in $\{x_1, \dots, x_d\}$.

7.5 Boosting Algorithm

Definition 7.5 (Boosting). Boosting is a weak learner slightly better than random guessing:

- **Weak learner:** slightly better than random guessing
- **Concentrate on hardest examples**
- **Weighted majority**

7.6 Adaboost

Definition 7.6 (Adaboost Algorithm). Training data $S = \{(x_1, y_1), \dots, (x_m, y_m)\}$. Initialize $D_1(i) = \dots = D_1(m) = \frac{1}{m}$.

For $t = 1, 2, \dots, T$ **do**:

- Train weak learner h_t to minimize weighted error: $\varepsilon_t = \sum_{i=1}^m D_t(i) \mathbb{I}[h_t(x_i) \neq y_i]$
- Compute weight: $\alpha_t = \frac{1}{2} \log \frac{1-\varepsilon_t}{\varepsilon_t}$
- Update distribution: $D_{t+1}(i) = \frac{D_t(i) e^{-\alpha_t y_i h_t(x_i)}}{Z_t}$ where $Z_t = \sum_i D_t(i) e^{-\alpha_t y_i h_t(x_i)}$

Final classifier: $H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$

At each round, misclassified examples receive higher weight (multiplied by e^{α_t}), forcing the weak learner to focus on harder examples.

7.7 Adaboost Training Error Bound

Theorem 7.1 (Adaboost Training Error). Given a training set $\{(x_1, y_1), \dots, (x_m, y_m)\}$ and assume each weak learner in Adaboost has weighted error $2\varepsilon_t \leq 1 - D$. Then training error is at most:

$$\frac{1}{m} \sum_{i=1}^m \mathbb{I}[\mathcal{H}(x_i) \neq y_i] \leq \exp(-2\gamma^2 T) \quad (\text{Training error drops exponentially fast})$$

7.8 Training Error Analysis

Theorem 7.2 (Adaboost Training Error Decomposition). Let the training error be:

$$\frac{1}{m} \sum_{i=1}^m \mathbb{I}[\mathcal{H}(x_i) \neq y_i] \leq \frac{1}{m} \sum_{t=1}^T e^{-y_i f(x_i)} = \sum_{t=1}^T z_t$$

Where $f(x_i) = \sum_k \alpha_k h_k(x_i)$. If $\mathcal{H}(x_i) = \text{sign}(f(x_i))$ and $y_i \neq \text{sign}(f(x_i))$, then $-y_i f(x_i) > 0 \Rightarrow e^{-y_i f(x_i)} \geq 1$.

Second equality: $D_t(i) = \frac{D_t(i) e^{-\alpha_t y_i h_t(x_i)}}{z_t}$ where:

$$z_t = \sum_{i=1}^m e^{-y_i f(x_i)}$$

Thus $e^{-y_i f(x_i)} = e^{-y_i f(x_{i-1})} \cdot e^{-\alpha_t y_i h_t(x_i)} = \prod_{t=1}^T \frac{D_t(i)}{D_t(i)} z_t = \frac{D_T(i)}{D_1(i)} z_t$.

So $\frac{1}{m} \sum e^{-y_i f(x_i)} = \left(\prod_{p=1}^T D_p(i) \right) \left(\prod_{t=1}^T z_t \right) = \prod_t z_t$.

To minimize training error, we can minimize z_t :

$$\frac{\partial}{\partial z_t} z_t e^{-\alpha_t y_i h_t(x_i)} = e^{\alpha_t \sum D_t(i) + e^{-\alpha_t} \sum D_t(i) = \frac{1}{z_t e^{\alpha_t}}} \sqrt{(1 - 4\gamma_t^2)}$$

Take $\frac{\partial z_t}{\partial \alpha_t} = 0 \Rightarrow \alpha_t = \frac{1}{2} \log \frac{1-\varepsilon_t}{\varepsilon_t}$ and $z_t = 2\sqrt{\varepsilon_t(1-\varepsilon_t)} = \sqrt{1-4\gamma_t^2}$ (where $\gamma_t = \frac{1}{2} - 2\varepsilon_t$).

So $\frac{1}{m} \sum \mathbb{I}[\mathcal{H}(x_i) \neq y_i] \leq \prod_t z_t = \prod_t \sqrt{1-4\gamma_t^2} \leq e^{-2\sum \gamma_t^2}$ (last inequality we use $1-x \leq e^{-x}$).

7.9 Boosting as Forward Stagewise Additive Model

Definition 7.7 (Forward Stagewise Additive Model). Boosting is a greedy way to solve the problem:

$$\min \left\{ \sum_{i=1}^m V(y_i, \sum_t \alpha_t h_t(x_i)) : \alpha_1, \dots, \alpha_T \in \mathbb{R}^T, h_1, \dots, h_T \in \mathcal{H}^T \right\}$$

At each iteration, $f^{(t)} = \sum_{s=1}^t \alpha_s h_s$ and $(\alpha_t, h_t) = \arg \min_{\alpha, h} \sum_i V(y_i, f^{(t-1)}(x_i) + \alpha h_t(x_i))$.

This is called forward stagewise additive model.

To derive adaboost, we denote $V(y, \hat{y}) = \exp(-y\hat{y})$ and minimize $\sum_i e^{-y_i f^{(t)}(x_i) + \alpha h_t(x_i)}$. We have $\min \sum_i D_t(i) \mathbb{I}[y_i \neq h_t(x_i)]$ (weighted classification error).

So h_t minimizes the weighted classification error h_t minimizes $\sum_i D_t(i) \mathbb{1}[y_i \neq h_t(x_i)]$ (weighted classification error).

α_t, h_t can be minimized separately. $\alpha_t = \frac{1}{2} \log \frac{1-\varepsilon_t}{\varepsilon_t}$, ε_t is the weighted classification error h_t minimizes.

So exponential loss $\Leftrightarrow$ weighted 0-1 classification error.

7.10 Typical Loss Functions

Definition 7.8 (Classification Loss Functions). Common loss functions are $V_{\text{clf}}(y, \hat{y}) = \mathbb{1}[y \neq \text{sign}(\hat{y})]$ (misclassification loss):

- **Misclassification loss:** $V_{\text{clf}}(y, \hat{y}) = \mathbb{1}[y \neq \text{sign}(\hat{y})]$
- **Hinge loss:** $V_{\text{hinge}}(y, \hat{y}) = \max(0, 1 - y\hat{y})$
- **Exponential loss:** $V_{\text{exp}}(y, \hat{y}) = \exp(-y\hat{y})$
- **Square loss:** $V_{\text{sq}}(y, \hat{y}) = (y - \hat{y})^2$

Key properties:

- **Margin of a predictor:** $\hat{y}$ for $y \in \{-1, 1\}$
- **Misclassification is not continuous**
- **Square loss punishes positive margin unnecessarily**
- **Hinge loss only punishes negative margin but not differentiable everywhere**
- **Exponential loss punishes negative margin and promotes large positive margins**

7.11 Bagging vs Boosting

Remark 7.1 (Variance vs Bias Reduction). Bagging tends to reduce variance. Boosting tends to reduce bias.

8 Theoretical Foundations: Empirical Risk and Generalization

This section develops the theoretical framework for understanding when and why empirical risk minimization (ERM) succeeds, establishing key results about generalization bounds and consistency.

8.1 Empirical Risk and Its Properties

Definition 8.1 (Empirical Risk). Consider a loss function $\ell : \mathcal{Z} \times \mathcal{Y} \rightarrow \mathbb{R}$. The empirical risk is:

$$\mathcal{E}_n(f) = \frac{1}{n} \sum_{i=1}^n \ell(f(x_i), y_i)$$

which is an unbiased estimator of the true risk $\mathcal{E}(f) := \mathbb{E}_{(x,y) \sim P}[\ell(f(x), y)]$ with variance $V_\ell = \frac{\text{Var}(\ell(f(x), y))}{n}$.

Remark 8.1 (Variance Bound). The variance of the empirical risk estimator satisfies:

$$\mathbb{E}[|\mathcal{E}_n(f) - \mathcal{E}(f)|] \leq \sqrt{\frac{V_\ell}{n}}$$

Definition 8.2 (Chebyshev's Inequality). For any random variable X with mean μ and standard deviation σ , and any $k > 0$:

$$\mathbb{P}(|X - \mu| > k\sigma) \leq \frac{1}{k^2}$$

This gives the probability that X deviates beyond k standard deviations.

8.2 Excess Risk Decomposition

Definition 8.3 (Excess Risk Decomposition). Let f^* be the optimal function on the infinite population and f_n be the empirical risk minimizer over hypothesis class $\mathcal{F}$. The excess risk can be decomposed as:

$$\mathcal{E}(f_n) - \mathcal{E}(f^*) = \underbrace{\mathcal{E}(f^*) - \mathcal{E}(f)}_{\text{approximation error}} + \underbrace{\mathcal{E}(f) - \mathcal{E}_n(f_n)}_{\text{generalization error}} + \underbrace{\mathcal{E}_n(f_n) - \mathcal{E}(f_n)}_{\text{estimation error}}$$

where $f = \arg \min_{f \in \mathcal{F}} \mathcal{E}(f)$ is the best function in the hypothesis class.

Definition 8.4 (Generalization Error). The generalization error measures the gap between empirical and true risk:

$$\text{Generalization Error} = \mathbb{E}[\mathcal{E}(f_n) - \mathcal{E}_n(f_n)]$$

In general $\mathbb{E}[\mathcal{E}_n(f_n) - \mathcal{E}(f_n)] \neq 0$ because both $\mathcal{E}_n$ and f_n depend on the training data. Specifically, for samples $X_i = \ell(f_n(x_i), y_i)$ and $X_j = \ell(f_n(x_j), y_j)$, the terms X_i and X_j are not independent.

Definition 8.5 (Overfitting). An estimator f_n is said to overfit the training data if for all $n \in \mathbb{N}$:

$$\mathbb{E}[\mathcal{E}(f_n) - \mathcal{E}_n(f_n)] > C \quad \text{and} \quad \mathbb{E}[\mathcal{E}(f_n)] \text{ decreases}$$

In other words, f_n performs worse on the true distribution but better on the training sample. This occurs when f_n learns patterns specific to the training data rather than general structure.

8.3 Empirical Risk Minimization on Finite Hypothesis Spaces

Definition 8.6 (ERM on Finite Hypothesis Spaces). When both $\mathcal{X}$ and $\mathcal{Y}$ are finite, the hypothesis class $\mathcal{F} = \mathcal{Y}^{\mathcal{X}} = \{f : \mathcal{X} \rightarrow \mathcal{Y} \mid \mathcal{X}, \mathcal{Y} \text{ finite}\}$ is also finite. For the empirical risk minimizer on $\mathcal{F}$:

$$\mathbb{E}[|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)|] = \mathbb{E}\left[\sup_{f \in \mathcal{F}} |\mathcal{E}_n(f) - \mathcal{E}(f)|\right] \leq |\mathcal{F}| \sqrt{\frac{V_f}{n}}$$

where $V_f = \sup_f V_f$ represents the maximum variance over all functions in $\mathcal{F}$.

Remark 8.2 (ERM Consistency). Here ERM works effectively as $\lim_{n \rightarrow \infty} \mathbb{E}[|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)|] = 0$.

Since $\mathcal{E}_n(f_n) - \mathcal{E}(f_n) \leq 0$ (because $f_n = \arg \min \mathcal{E}_n(f)$), we have:

$$\mathbb{E}[\mathcal{E}(f_n) - \mathcal{E}(f_n)] \leq \mathbb{E}[\mathcal{E}(f) - \mathcal{E}_n(f)] \leq 2 \sup_{f \in \mathcal{F}} |\mathcal{E}_n(f) - \mathcal{E}(f)|$$

Theorem 8.1 (Generalization Bound for Finite Hypothesis Spaces). Let $\mathcal{H} \subseteq \mathcal{F}$ be a finite space of hypotheses. Then:

$$\mathbb{E}[|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)|] \leq |\mathcal{H}| \sqrt{\frac{V_f}{n}}$$

In particular, if $f_n \in \mathcal{H}$, then $\mathbb{E}[|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)|] \leq |\mathcal{H}| \sqrt{\frac{V_f}{n}}$.

The expected generalization error vanishes as $n \rightarrow \infty$, and the expected excess risk also vanishes. Thus, on finite hypothesis spaces, ERM is consistent.

8.4 Hoeffding's Inequality and High-Probability Bounds

Definition 8.7 (Hoeffding's Inequality). Let $X_1, \dots, X_n$ be independent random variables with $X_i \in [a_i, b_i]$. Let $\bar{X} = \frac{1}{n} \sum_i X_i$. Then:

$$\mathbb{P}(|\bar{X} - \mathbb{E}[X]| > \varepsilon) \leq 2 \exp\left(-\frac{2n^2\varepsilon^2}{\sum_i (b_i - a_i)^2}\right)$$

Definition 8.8 (Bounded Loss Condition). If $|\ell(f(x), y)| \leq M$ by some constant $M > 0$, then for all $f \in \mathcal{H}$:

$$\mathbb{P}(|\mathcal{E}_n(f) - \mathcal{E}(f)| > \varepsilon) \leq 2 \exp\left(-\frac{2n\varepsilon^2}{M^2}\right)$$

8.5 Union Bound and Sample Complexity

Definition 8.9 (Union Bound for Generalization).

$$\begin{aligned} \mathbb{P}(|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)| > \varepsilon) &\leq \mathbb{P}\left(\sup_{f \in \mathcal{F}} |\mathcal{E}_n(f) - \mathcal{E}(f)| \geq \varepsilon\right) \\ &\leq \sum_{f \in \mathcal{F}} \mathbb{P}(|\mathcal{E}_n(f) - \mathcal{E}(f)| \geq \varepsilon) \leq 2|\mathcal{H}| \exp\left(-\frac{n\varepsilon^2}{2M^2}\right) \end{aligned}$$

For any $\delta \in (0, 1]$, with probability at least $1 - \delta$:

$$|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)| \leq \sqrt{\frac{2M^2 \log(2|\mathcal{H}|/\delta)}{n}}$$

Theorem 8.2 (Convergence Rate). The rate of convergence of the generalization error is $O(1/\sqrt{n})$.

8.6 Gap When True Function Outside Hypothesis Class

Remark 8.3 (Irreducible Gap). If the optimal function $f^* \notin \mathcal{H}$, then ERM on $\mathcal{H}$ will never minimize the expected risk. There is always an irreducible gap. Consider $f_n = \arg \min_{f \in \mathcal{H}} \mathcal{E}_n(f)$ with $f^* \in \mathcal{H}^c$. The excess risk satisfies:

$$\mathcal{E}(f_n) - \mathcal{E}(f^*) \leq |\mathcal{E}(f^*) - \mathcal{E}(f_n)| + (\text{estimation and generalization error})$$

This demonstrates the importance of choosing an expressive enough hypothesis class to contain or approximate the true function.

9 Rademacher Complexity and Advanced Generalization Bounds

This section develops the theory of Rademacher complexity, a fundamental tool for deriving data-dependent generalization bounds that adapt to hypothesis class complexity.

9.1 Regularization and Excess Risk Decomposition

Definition 9.1 (Regularization Framework). σ is a regularization parameter. Assume $\mathcal{H} \subseteq \mathcal{L}_\sigma$ where $|\gamma| \leq \sigma$. Given training points, we find an estimator $f_{n,n}$ on $\mathcal{H}_\sigma$. Let $\gamma = \sigma(n)$ increase as $n \rightarrow \infty$.

Definition 9.2 (Excess Risk Decomposition with Regularization). For $f_n = \arg \min_{f \in \mathcal{H}} \mathcal{E}_n(f)$, the excess risk can be decomposed as:

$$\mathcal{E}(f_{n,n}) - \mathcal{E}(f^*) = \underbrace{\mathcal{E}(f_{n,n}) - \mathcal{E}_n(f_{n,n})}_{\text{sample error}} + \underbrace{\mathcal{E}_n(f_{n,n}) - \inf_{f \in \mathcal{H}} \mathcal{E}(f)}_{\text{approximation error}} + \underbrace{\inf_{f \in \mathcal{H}} \mathcal{E}(f) - \mathcal{E}(f^*)}_{\text{irreducible error}}$$

Definition 9.3 (Universal Hypothesis Classes). If the irreducible error is 0, then $\mathcal{H}$ is called *universal*. For example, RBF (Radial Basis Functions) induced by Gaussian kernels form a universal hypothesis class.

Definition 9.4 (Approximation Error (Bias)). Approximation error does not depend on the dataset but depends on the underlying distribution p . It is also referred to as *bias*. Under mild assumptions:

$$\lim_{n \rightarrow \infty} \mathbb{E}[\mathcal{E}(f_n) - \inf_{f \in \mathcal{H}} \mathcal{E}(f)] = 0$$

Definition 9.5 (Sample Error). Sample error is a random quantity depending on data:

$$\mathcal{E}(f_{n,n}) - \mathcal{E}(f^*) = \underbrace{\mathcal{E}(f_{n,n}) - \mathcal{E}_n(f_{n,n})}_{\text{generalization error}} + \underbrace{\mathcal{E}_n(f_{n,n}) - \mathcal{E}_n(f^*)}_{\text{excess empirical risk}} + \underbrace{\mathcal{E}(f^*) - \mathcal{E}(f^*)}_{\text{generalization error, } O(1) \text{ in expectation}}$$

9.2 Rademacher Complexity

Definition 9.6 (Empirical Rademacher Complexity). Let $\mathcal{Z}$ be a set and $S = \{z_i\}_{i=1}^n$ a dataset on $\mathcal{Z}$. The empirical Rademacher complexity of a space of hypotheses $\mathcal{H} \subseteq \{f : \mathcal{Z} \rightarrow \mathbb{R}\}$ is:

$$\mathcal{R}_S(\mathcal{H}) = \mathbb{E}_\sigma \left[\sup_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i f(z_i) \right]$$

where $\sigma_i = \{\sigma_i\}_{i=1}^n$ are Rademacher variables with $\mathbb{P}(\sigma_i = 1) = \mathbb{P}(\sigma_i = -1) = \frac{1}{2}$.

Definition 9.7 (Population Rademacher Complexity). The Rademacher complexity is the expectation of $\mathcal{R}_S(\mathcal{H})$ over all samples:

$$\mathcal{R}_n(\mathcal{H}) = \mathbb{E}_{S \sim \rho^n} [\mathcal{R}_S(\mathcal{H})]$$

Remark 9.1 (Intuition of Rademacher Complexity). Rademacher complexity measures how well a hypothesis class can fit pure noise. If $\mathcal{H}$ is powerful, there exists an f that correlates with many random ± 1 signals, making the supremum in the definition large (high complexity). Formally:

$$\mathcal{R}_S(\mathcal{H}) \approx \text{“average maximum correlation with random } \pm 1 \text{ noise”}$$

Smaller Rademacher complexity values guarantee smaller generalization gaps, while large values signal potential overfitting.

9.3 Generalization Bounds via Rademacher Complexity

Theorem 9.1 (Rademacher Complexity Bound).

$$\mathbb{E}_S \left[\sup_{f \in \mathcal{H}} |\mathcal{E}(f) - \mathcal{E}_n(f)| \right] \leq 2\mathcal{R}_n(\ell \circ \mathcal{H})$$

where $\ell : \mathcal{Z} \times \mathcal{Y} \rightarrow \mathbb{R}$ is a loss function with $\ell(f(x), y)$.

Remark 9.2 (Proof Sketch). The proof uses the symmetry argument:

$$\begin{aligned} \mathbb{E}_S [\sup_{f \in \mathcal{H}} |\mathcal{E}(f) - \mathcal{E}_S(f)|] &= \mathbb{E}_S \left[\sup_{f \in \mathcal{H}} |\mathbb{E}_{s, \sigma} [\sigma_i \ell(f)] - \mathbb{E}_S [\sigma_n \ell(f)]| \right] \\ &\leq \mathbb{E}_{S, S'} \left[\sup_{f \in \mathcal{H}} |\sigma_{s, i} \ell_i(f) - \ell'_i(f)| \right] \\ &= \mathbb{E}_{S, S'} \left[\sup_{f \in \mathcal{H}} |\sigma(\ell_{s, i}(f) - \ell_{s', i}(f))| \right] \\ &\leq 2\mathbb{E}_{S, \sigma} \left[\sup_{f \in \mathcal{H}} \frac{1}{n} \sum_i \sigma_i \ell(f(x_i), y_i) \right] = 2\mathcal{R}_n(\ell \circ \mathcal{H}) \end{aligned}$$

9.4 Lipschitz Contraction and Bounded Losses

Theorem 9.2 (Contraction Principle). Let $\ell(\cdot, y)$ be L -Lipschitz uniformly for all $y \in \mathcal{Y}$ with $L > 0$. Then, for any set $S = \{(x_i, y_i)\}_{i=1}^n$:

$$\mathcal{R}_S(\ell \circ \mathcal{H}) \leq L\mathcal{R}_S(\mathcal{H}), \quad \text{with } S_\mu = \{X_i\}_{i=1}^n$$

Furthermore, for any probability distribution ρ on $\mathcal{X} \times \mathcal{Y}$ and any $n \in \mathbb{N}$:

$$\mathcal{R}_n(\ell \circ \mathcal{H}) \leq L\mathcal{R}_n(\mathcal{H})$$

This shows that Lipschitz loss functions preserve and contract Rademacher complexity.

10 McDiarmid Inequality, RKHS, and Optimization Methods

The previous section developed deterministic generalization bounds. This section introduces probabilistic concentration inequalities and connects theory to practical optimization algorithms.

10.1 McDiarmid's Inequality and High-Probability Bounds

Theorem 10.1 (McDiarmid's Inequality). Let $\mathcal{Z}$ be a set and $g : 2^n \rightarrow \mathbb{R}$ be a function such that there exists $c > 0$ such that for all $i = 1, \dots, n$ and for all $z_1, \dots, z_n, z'_i \in \mathcal{Z}$:

$$|g(z_1, \dots, z_n) - g(z_1, \dots, z_{i-1}, z'_i, z_{i+1}, \dots, z_n)| \leq c$$

Let $Z_1, \dots, Z_n$ be n independent random variables taking values in $\mathcal{Z}$. Then for all $\delta > 0$, with probability at least $1 - \delta$:

$$|g(Z_1, \dots, Z_n) - \mathbb{E}[g(Z_1, \dots, Z_n)]| \leq c \sqrt{\frac{n \log \frac{2}{\delta}}{2}}$$

Remark 10.1 (Application to Empirical Risk). Let $z_i = (x_i, y_i)$ and

$$g(Z_1, \dots, Z_n) = \sup_{f \in \mathcal{H}} \left[\mathcal{E}(f) - \frac{1}{n} \sum_{i=1}^n \ell(f(x_i), y_i) \right]$$

Assume $\ell(\cdot, y) \in \mathcal{L}^c$. Then:

$$|g(Z_1, \dots, Z_n) - g(Z_1, \dots, Z_{i-1}, z'_i, z_{i+1}, \dots, Z_n)| \leq \frac{1}{n} |\ell(f(x), y_1) - \ell(f(x), y_2)| \leq \frac{2c}{n}$$

Thus, for all $\delta > 0$ with probability at least $1 - \delta$:

$$\begin{aligned} \sup_{f \in \mathcal{H}} |\mathcal{E}(f) - \mathcal{E}_n(f)| &\leq \mathbb{E}_S \left[\sup_{f \in \mathcal{H}} |\mathcal{E}(f) - \mathcal{E}_n(f)| \right] + c \sqrt{\frac{2 \log(2/\delta)}{n}} \\ &\leq 2L\mathcal{R}_n(\mathcal{H}) + c \sqrt{\frac{2 \log(2/\delta)}{n}} \end{aligned}$$

10.2 Gradient Descent Convergence

Theorem 10.2 (Gradient Descent Error Bound).

$$\mathbb{E}[\mathcal{E}_n(f_n) - \mathcal{E}_n(f_n^*)] < O(1/\sqrt{k})$$

for gradient descent after k iterations.

10.3 Reproducing Kernel Hilbert Space (RKHS) and Bounded Kernels

Definition 10.1 (RKHS with Bounded Kernels). In RKHS where $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is a bounded kernel with $k(x, x) \leq x^2$. Let $\mathcal{H}$ be the RKHS associated with k and $\mathcal{H}_\gamma$ be the space of functions $f \in \mathcal{H}$ such that $\|f\|_k \leq \sigma$. Then:

$$\mathcal{R}(\mathcal{H}_\gamma) \leq \frac{\sigma\sqrt{k}}{\sqrt{n}}$$

As $n \rightarrow \infty$, we have $\mathcal{R}(\mathcal{H}_\gamma) \rightarrow 0$, and as $\sigma \uparrow$, less regularization is required.

Remark 10.2 (Tikhonov Regularization Connection). In Tikhonov regularization, $\|w_{s,\lambda}\| \leq \frac{M}{\sqrt{\lambda}}$. If $\ell(y, y') \leq M^2$, we can restrict to $\mathcal{H}_\gamma$ with $\gamma = \frac{M}{\sqrt{\lambda}}$.

10.4 Ivanov Regularization

Definition 10.2 (Ivanov Regularization). Ivanov regularization solves:

$$w_{s,\sigma} = \arg \min_{\|w\| \leq \sigma} \mathcal{E}_S(w)$$

10.5 Projected Gradient Descent (PGD)

Definition 10.3 (Smooth Convex Optimization with Constraints). Let $F : \mathbb{R}^d \rightarrow \mathbb{R}$ be a smooth closed convex function and $\mathcal{C} \subseteq \mathbb{R}^d$ be a convex set. We seek to solve the constrained optimization problem:

$$\min_{w \in \mathcal{C}} F(w)$$

Definition 10.4 (Projected Gradient Descent). A variant of gradient descent that respects convex constraints. Let $\Pi_{\mathcal{C}}(w) = \arg \min_{z \in \mathcal{C}} \|z - w\|^2$ be the projection of w onto $\mathcal{C}$. Then:

$$w_{s,t} = \Pi_{\mathcal{C}}(w_{s,t} - \nabla F(w_{s,t}))$$

Remark 10.3 (Ivanov Projection). In Ivanov regularization, the projection operator is:

$$\Pi_{\mathcal{B}_\sigma}(w) = \begin{cases} w & \text{if } \|w\| \leq \sigma \\ \frac{w}{\|w\|}\sigma & \text{otherwise} \end{cases}$$

Theorem 10.3 (PGD Convergence Rate). If $\eta = 1/M$ for a convex and M -smooth function F , then:

$$F(w_{s,t}) - F(w^*) \leq \frac{2M}{t^2} \|w_s - w^*\|^2$$

showing quadratic (geometric) convergence after t iterations.

10.6 Introduction to Online Learning

Definition 10.5 (Online Learning Model). • **Data:** Online sequence of data (usually with no distributional assumptions)

- **Aim:** Sequentially predict and update a classifier to predict well on the sequence, with no assumption of training/testing distribution separation
- **Evaluation Metric:** Cumulative error over all predictions

11 Online Learning with Experts

This section studies the problem of combining predictions from multiple experts in an online setting, where the learner makes sequential predictions and receives feedback.

11.1 Online Learning with Experts Framework

Definition 11.1 (Expert Prediction Setup). Let $S = S_m = \{(x_1, y_1), \dots, (x_m, y_m)\}$ with $(x_t, y_t) \in [0, 1]^n \times [0, 1]$. Here:

- x_t is a vector of predictions from n experts about an outcome y_t
- Goal: Find a master algorithm A that combines the expert predictions

The evaluation metric is the loss (mistakes) of the master algorithm A on S :

$$L_A(S) := \sum_{t=1}^m |y_t - \hat{y}_t|$$

where $\hat{y}_t = A(x_1, \dots)(x_t)$ is the prediction of online algorithm A trained on $S_{\leq t}$ and evaluated on x_t .

Definition 11.2 (Halving Algorithm). For any sequence with one consistent expert (always right), the Halving algorithm makes at most $\log_2 n$ mistakes. Whenever the algorithm makes a mistake, it eliminates more than half of the remaining experts.

Definition 11.3 (Expert Loss). The loss of the i -th expert E_i is:

$$L_i(S) = \sum_{t=1}^m |y_t - x_{t,i}|$$

11.2 Weighted Majority Algorithm

Definition 11.4 (Weighted Majority Algorithm (WMA)).

- **Majority Rule:** If the majority is right, the weights of minority experts are multiplied by β . If the majority makes a mistake, multiply the majority weights by β .

- **Weight Updates:**

$W_{t,i} = \beta^{M'_i}$, where $W_{t,i}$ is the weight of E_i at the beginning of trial t and M'_i is the number of mistakes of E_i at the start of trial t .

- **Total Weight:**

$$W_{t+1} \leq \beta E W_t, \quad W_t = \sum_{i=1}^m W_{t,i} \text{ (total weight at the start of trial } t)$$

- **Bound Derivation:**

$$W_{t+1} \leq \left(\frac{1-\beta}{2}\right)^T W_i = \left(\frac{1-\beta}{2} E\right)^n$$

and

$$W_{t+1} = \sum_{j=1}^m W_{t,j} = \sum_{j=1}^m \beta^{M_j} \geq \beta^{M^*} \text{ where } M^* \text{ is the best expert's mistakes}$$

Theorem 11.1 (Weighted Majority Bound). Combining the above, we get:

$$T \leq \frac{\ln \beta}{\ln \frac{(1-\beta)E}{2}} M_* + \frac{1}{\ln(1/\beta)} \ln n$$

Since the total mistakes the master algorithm makes is $M \leq T$:

$$M \leq \frac{\ln \beta}{\ln \frac{(1-\beta)E}{2}} M_* + \frac{1}{\ln(1/\beta)} \ln n$$

11.3 Loss Functions for Online Learning

Definition 11.5 (Entropic Loss). For a loss function $L : [0, 1] \times L_0([\cdot], 1) \rightarrow [0, +\infty)$, the entropic (logarithmic) loss is:

$$L(y, \hat{y}) = y \ln \hat{y}^{-1} + (1 - y) \ln(1 - \hat{y})^{-1}$$

- When $y = 0$: $L = -\log(\hat{y})$
- When $y = 1$: $L = -y \ln \hat{y}$

This measures the cross entropy between the true label and the model's distribution. Here Q is the true label and P_0 is the model prediction.

11.4 Weighted Average Algorithm

Definition 11.6 (Weighted Average (WA) Algorithm). **Initialize:** $v_i = (\frac{1}{n}, \dots, \frac{1}{n})$, $L_{WA} := 0$, $L := 0$. Let $\eta \in (0, \infty)$ be the learning rate with $\eta = -\log \beta$.

For $t = 1, \dots, m$ **Do:**

1. **Receive instance:** $x_t \in [0, 1]^n$
2. **Predict:** $\hat{y}_t = v_t \cdot x_t \in [0, 1]$
3. **Receive label:** $y_t \in \{0, 1\}$
4. **Calculate loss:** $L_{WA} += L(y_t, \hat{y}_t)$, $L_t := L(y_t, x_{t,i})$
5. **Update weights:**

$$v_{t+1,i} = \frac{v_{t,i} e^{-\eta L_t}}{\sum_j v_{t,j} e^{-\eta L_t}}$$

Theorem 11.2 (Weighted Average Algorithm Bound). For a sequence $S = (x_1, y_1), \dots, (x_m, y_m)$ with $(x_i, y_i) \in [0, 1]^n \times [0, 1]$:

$$L_{WA}(S) - \min_i L_i(S) \leq \frac{1}{\eta} \ln(n)$$

For specific losses:

- **Entropic loss:** $\eta = 1$
- **Square loss:** $\eta = 1/2$

Remark 11.1 (Algorithm Analysis via Potential Function). Define the potential function:

$$\text{Len}(y_t, \hat{y}_t) = \sum_{i=1}^m u_i \text{Len}(y_t, x_{t,i}) = d(u, y_t) - d(u, v_{t+1})$$

where the KL divergence $d(u, v_i) = \sum_i u_i \log \frac{u_i}{v_i}$ satisfies:

$$\sum_{t=1}^m \text{Len}(y_t, \hat{y}_t) - \sum_{t=1}^m \sum_{i=1}^m u_i \text{Len}(y_t, x_{t,i}) = d(u, y_t) - d(u, y_{m+1})$$

$$d(u, v_i) = \sum_i u_i \log \frac{u_i}{v_i} = \sum_i u_i \log u + \log n \leq \log n$$

This shows the cumulative loss is bounded by $\log n$, independent of the sequence length.

12 Advanced Online Learning Algorithms

This section covers advanced algorithms for online learning: HEDGE, the Perceptron, and online gradient descent with various loss functions.

12.1 HEDGE Algorithms

Definition 12.1 (HEDGE-1 Algorithm). Initialize $V_t \in \mathcal{L}_n$, receive predictions $x_t \in [0, 1]^n$.

- **Loss:** $L_t^{HA} = v_t \cdot x_t$, with $L_t = x_t$
- **Update:**

$$v_{t+1,i} := \frac{v_{t,i} e^{-\eta L_t}}{\sum_j v_{t,j} e^{-\eta L_t}}$$

Definition 12.2 (HEDGE-2 Algorithm). Sample an expert based on distribution v_t , and use that expert's prediction as $\hat{y}_t$.

- **Loss:** $\mathbb{E}[L_t^{HA}] := \mathbb{E}[L_t^{HA}] + v_t \cdot x_t$, with $L_i += \xi_{t,i}$
- **Update:**

$$v_{t+1,i} := \frac{v_{t,i} e^{-\eta L_t}}{\sum_j v_{t,j} e^{-\eta L_t}}$$

Theorem 12.1 (HEDGE Regret Bound). Let $S = \ell_1, \dots, \ell_m \in [0, 1]^n$. The regret of the HEDGE HA-2 algorithm with learning rate $\eta = \sqrt{\frac{2 \ln n}{m}}$ is:

$$\mathbb{E}[L_{HA}(S)] - \min_i L_i(S) \leq \sqrt{2m \ln n}$$

12.2 Perceptron Algorithm

Definition 12.3 (Perceptron Setup and Assumptions). **Assumption:** Data is linearly separable by some margin T . Hence there exists a hyperplane with normal vector u such that:

1. $\|u\| = 1$
2. (x_t, y_t) with $\|x_t\| \leq R$ and $y_t \in \{+1, -1\}$
3. $y_t(x_t \cdot u) > T$ (margin condition)

Definition 12.4 (Perceptron Algorithm). **Initialize:** $w_1 = 0$; $M_t = 0$ (number of mistakes).

For $t = 1$ to m **do:**

1. **Predict:** $\hat{y}_t = \text{sign}(w_t \cdot x_t)$
2. **If mistake** (i.e., $\hat{y}_t y_t \leq 0$):
 - **Update:** $w_{t+1} = w_t + y_t x_t$; $M_{t+1} = M_t + 1$

3. Else:

- $w_{t+1} = w_t$; $M_{t+1} = M_t$

Mistake Bound: Number of mistakes is at most $\left(\frac{R}{T}\right)^2$.

Remark 12.1 (Perceptron Convergence Analysis). If $(w_t \cdot x_t)y_t < 0$, then:

$$\|w_{t+1}\|^2 \leq \|w_t\|^2 + \|x_t\|^2 \leq M_t R^2, \quad \|w_t\| \geq M_t T$$

This gives the quadratic mistake bound.

12.3 Online Gradient Descent with Hinge Loss

Definition 12.5 (Online GD with Hinge Loss and ℓ^2 Penalty).

$$\begin{aligned} w_{t+1} &= \arg \min_{w \in H} L_{\text{hinge}}(y_t, w \cdot x_t) + \lambda \|w - w_t\|^2 \\ &= \arg \min_{w \in H} \max(-1 - y_t w \cdot x_t, 0) + \lambda \|w - w_t\|^2 \end{aligned}$$

This yields:

$$w_{t+1} = \begin{cases} Sw_t & \text{if } y_t(w \cdot x_t) > 1 \\ w_t + y_t x_t & \text{if } y_t(w \cdot x_t) < 1 \\ w_t - \lambda & \text{otherwise} \end{cases}$$

This algorithm can be generalized to other losses as long as the loss function is convex and its subgradient is bounded.

Definition 12.6 (OGD Algorithm Details). **Initialize:** $w^1 = 0$, $L_{\text{sqn}} = 0$, $\eta = \frac{1}{2\lambda}$

For $t = 1$ to m **do:**

1. $L_{\text{sqn}} += L_t(y_t, \hat{y}_t)$
2. $w_{t+1} = w_t + \eta u_t y_t \hat{y}_t x_t = w_t - \eta Z_t$

where Z_t is a subgradient of the hinge loss.

Theorem 12.2 (OGD Regret Bound). Let $R = \max_t \|x_t\|$ and $\eta = \frac{U}{\sqrt{Tm}}$. Then for any u with $\|u\| \leq U$, the regret produced by OGD satisfies:

$$\sum_{t=1}^m [L_{\text{hinge}}(y_t, \hat{y}) - L_{\text{hinge}}(y_t, u \cdot x_t)] \leq \sqrt{U^2 R^2 T m}$$

12.4 Surrogate Loss Approaches

Definition 12.7 (Surrogate Loss Framework). The surrogate loss approach consists of three stages:

- **Encoding** c : Map $Y \rightarrow \mathcal{H}$ into a suitable linear feature space $\mathcal{H}$ (e.g., $\mathcal{H} = \mathbb{R}^d$)

- **Surrogate Loss** L' : Define $L' : \mathcal{H} \times \mathcal{H} \rightarrow \mathbb{R}$ and learn a classifier $g_n : \mathcal{Z} \rightarrow \mathcal{H}$ over hypotheses $\mathcal{G}_n$:

$$g_n = \arg \min_{g \in \mathcal{G}_n} \frac{1}{n} \sum_{i=1}^n L(g_n(c(y_i)), c(y_i))$$

where the original problem is $\ell(f(x), y)$ with $f : \mathcal{X} \rightarrow \mathcal{Y}$

- **Decoding** d : Map $\mathcal{H} \rightarrow \mathcal{Y}$ such that the final model is $f_n = d \circ g_n : \mathcal{X} \rightarrow \mathcal{Y}$ with structure $h(\psi) = d(g(\psi))$

13 Multi-Class Classification and Surrogates

13.1 Multi-Class Classification

Definition 13.1 (Multi-Class Problem). For multi-class classification with T classes:

$$Y = \{1, 2, \dots, T\} \quad \text{for } T \text{ classes}$$

The encoding strategy is:

$$c : Y \rightarrow \mathbb{R}^T, \quad \text{one-hot vector } c(t) = e_t e^T \quad (\text{where } e_t \text{ is the vector of 0 but 1 in the } t\text{-th position})$$

The one-vs-all loss is:

$$L(g(x), e_j) = \sum_{t=1}^T L(g(x)^{(t)}, e_j^{(t)}) \quad \begin{array}{l} g(x) \in \mathbb{R}^T, \text{ where } e_j \in \mathbb{R}^T \\ g(x)^{(t)} \text{ is the } t\text{-th element} \\ L_t \text{ is the loss on } t\text{-th entry} \end{array}$$

The decoding function $d : \mathbb{R}^T \rightarrow Y$ such that:

$$d(g(x)) = \arg \max_{t \in \{1, \dots, T\}} g(x)^{(t)}$$

13.2 Finite Dimensional Loss as Matrix

Definition 13.2 (Matrix Representation of Multi-Class Loss). For $Y = \{1, 2, \dots, T\}$, we can write any loss $\ell : Y \times Y \rightarrow \mathbb{R}$ represented by a matrix $V \in \mathbb{R}^{T \times T}$:

$$V = \begin{bmatrix} z_{11} & z_{12} & \cdots & z_{1T} \\ z_{21} & z_{22} & \cdots & z_{2T} \\ \vdots & \vdots & \ddots & \vdots \\ z_{T1} & z_{T2} & \cdots & z_{TT} \end{bmatrix}$$

so that $\ell(y, z) = V_{yz}$ where e_y, e_z are one-hot vectors.

The prediction rule becomes:

$$f^*(x) = \arg \min_{z \in Y} \sum_{t=1}^T \ell(t, z) dP(Y = t | X = x) = \arg \min_{z \in Y} \sum_{t=1}^T e_t^T V e_z$$

where the optimal classifier is:

$$g_\ell(x) = \int e_y dP(Y|X)$$

13.3 Implicit Embeddings

Definition 13.3 (Implicit Embeddings). A continuous loss $\ell : Y \times Y \rightarrow \mathbb{R}$ admits an implicit embedding if there exists a map $C : Y \rightarrow \mathcal{H}$ into a separable Hilbert space $\mathcal{H}$ and a linear operator $V : \mathcal{H} \rightarrow \mathcal{H}$ such that:

$$\ell(z, y) = \langle C(z), V C(y) \rangle_{\mathcal{H}}$$

13.4 Using Implicit Embeddings

Remark 13.1 (Implicit Embedding Application). If ℓ has an implicit embedding, then:

$$f^*(x) = \arg \min_{z \in Y} \langle C(z), Vg_\ell(x) \rangle_{\mathcal{H}}$$

where:

$$g_\ell : \mathcal{X} \rightarrow \mathcal{H} \quad \text{such that} \quad g_\ell(x) = \int c(y) dP(Y|X)$$

the conditional mean embedding of $P(Y|X)$ w.r.t. $\ell(z, y) = \langle c(z), Vg_\ell(x) \rangle_{\mathcal{H}}$.

We can replace g_ℓ with an estimator $\hat{g}_n : \mathcal{Z} \rightarrow \mathcal{H}$:

$$\hat{f}_n(x) = \arg \min_{z \in Y} \langle c(z), V\hat{g}_n(x) \rangle_{\mathcal{H}}$$

13.5 General Surrogate Approach

Definition 13.4 (General Surrogate Approach). The general framework consists of three components:

- **Encoding** $c : Y \rightarrow \mathcal{H}$ maps outputs to abstract vector space
- **Surrogate Loss** $L' : \mathcal{H} \times \mathcal{H} \rightarrow \mathbb{R}$ defining a problem to learn a function $g_n : \mathcal{Z} \rightarrow \mathcal{H}$ over $\mathcal{G}$:

$$g_n = \arg \min_{g \in \mathcal{G}} \frac{1}{n} \sum_{i=1}^n \|(g(c(y_i)) - c(y_i))\|_{\mathcal{H}}^2$$

- **Decoding** $d : \mathcal{H} \rightarrow \mathcal{Y}$ such that $f_n = d \circ g_n : \mathcal{X} \rightarrow \mathcal{Y}$ where $f_n(x) = d(g_n(x))$

13.6 Kernel Feature Approximation

Remark 13.2 (Linear Approximation via Kernel). Let $\mathcal{Z} = \mathbb{R}^d$. We can approximate g_ℓ with $g_\ell(x) = W_\lambda x$ where:

$$W_\lambda = \arg \min_{W \in \mathcal{G}_{\text{grad}}} \frac{1}{n} \sum_{i=1}^n \|c(y_i) - Wx_i\|^2 + \lambda \|W\|_{\text{Frob}}^2$$

$$g_\ell(x) = W_\lambda x = Y^T (X(X^T X + \eta I)^{-1}) x \quad X \in \mathbb{R}^{n \times d}, \quad Y \in \mathbb{R}^{D \times n}$$

We can decompose this as:

$$\alpha(x) = (\alpha_F(x), \dots, \alpha_H(x))^T = [X(X^T X + \eta I)^{-1}] x \in \mathbb{R}^n$$

If we have kernel, then $\alpha(x) = (k + \eta I)^{-1} v(x)$ where $v(x) \in \mathbb{R}^n$ is the evaluation vector $v(x_i) = k(x_i, x)$.

$$W_\lambda = \arg \min_{w \in \mathcal{G}_{\text{grad}}} \frac{1}{n} \sum_{i=1}^n \|(c(y_i) - Wx_i)\|^2 + \lambda \|W\|_{\text{NS}}^2$$

14 Convergence Analysis and Consistency

14.1 Decoding in Multi-Class

Definition 14.1 (Decoding Function). The decoding function $d : \mathcal{H} \rightarrow Y$ is such that:

$$f_n = d \circ g_n \quad \text{with} \quad d \circ g = \arg \min_{z \in Y} \langle c(z), Vg_\ell(x) \rangle_{\mathcal{H}}$$

This gives:

$$\begin{aligned} f_n(x) &= \arg \min_{z \in Y} \langle c(z), Vg_n(x) \rangle = \arg \min_{z \in Y} \langle c(z), V \left(\sum_i \alpha_i(x) c(y_i) \right) \rangle \\ &= \arg \min_{z \in Y} \sum_{i=1}^n \sum_{t=1}^T \alpha_i(x) \ell(z, y_i) \end{aligned}$$

We only need α and how to minimize ℓ over y .

14.2 Implicit Embeddings Condition

Lemma 14.1 (IE (Implicit Embeddings) Condition). Let ℓ admit an IE. Let $\Omega = \sup_y \|c(y)\|_{\mathcal{H}}$ and $\|V\| = \sup_g \|Vg\|_{\mathcal{H}}$, then:

$$\mathcal{E}(\ell \circ \text{dog}) - \mathcal{E}(\ell \circ f_x) \leq 2\Omega \|V\| \sqrt{\mathcal{R}(g_{\text{yes}}) - \mathcal{R}(\tilde{g}_*)}$$

If $\mathcal{R}(g_n) \rightarrow \mathcal{R}(\ell_k^*)$, then $\mathcal{E}(\ell \circ \tilde{g}_n) \rightarrow \mathcal{E}(\ell f_x)$.

14.3 Universal Consistency

Theorem 14.1 (Universal Consistency). Let $\mathcal{Z}, Y$ be compact. Let ℓ admit an implicit embedding and $k : \mathcal{Z} \times \mathcal{Z} \rightarrow \mathbb{R}$ be a universal kernel. Choose $\lambda = \frac{1}{n^\alpha}$ to train f_n , then:

$$\lim_{n \rightarrow \infty} \mathbb{E}[\mathcal{E}(f_n)] = \mathcal{E}(f^*) \quad \text{with probability 1}$$

14.4 Learning Rates

Remark 14.1 (Learning Rates for Surrogate Losses). For $\delta \in (0, 1)$:

$$\mathcal{E}(f_n) - \mathcal{E}(f^*) = \mathcal{O}(\|V\| \log(1/\delta) n^{-1/2})$$

hold with probability $1 - \delta$.